ИТМО

Аналитические БД

Аналитические и транзакционные БД

- Транзакционая / аналитическая
- Колоночная / строковая

Vertica

- Поддерживает: CSV, JSON, Parquet, Avro, ORC
- Может напрямую читать из Kafka, S3, HDFS
- Не требует администрирования

Недостатки

Текст, видео, картинки, звук

Запись данных

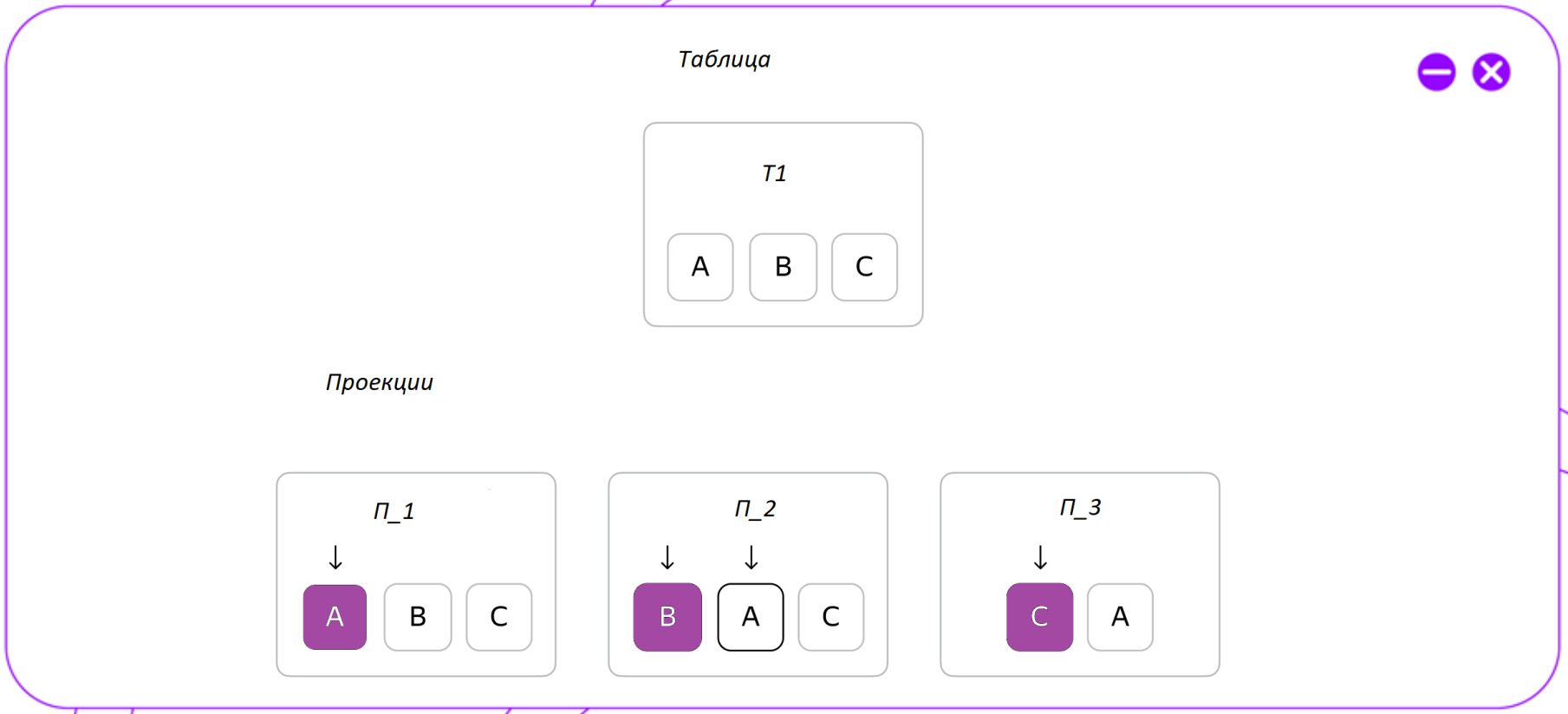
* CREATE TABLE shema_name.First_table (i int primary key, v varchar(16));

* ALTER TABLE shema_name.First_table SET MERGEOUT 0;

```
with vertica_python.connect(**conn_info) as first_connection:
curs = first_connection.cursor()
INSERT INTO shema_name.First_table VALUES ({},\'a\');
```

ERROR, Message: Too many ROS containers exist for the following projections…..

Как Vertica хранит данные



Как писать в Vertica

```
ALTER TABLE shema_name.First_table SET MERGEOUT 1;
```

```
COPY shema_name.First_table (i, v)
DELIMITER FROM LOCAL '/lessons/dwh/vertica/test_dataset.csv'
DELIMITER '|' ;

/* для атрибута */
COPY COPY_EXCEL1 (i, v ENFORCELENGTH )
FROM LOCAL 'lessons/dwh/vertica/test_dataset..csv' DELIMITER '|';

/* для всех строковых атрибутов */
COPY COPY_EXCEL1 (i, v)
FROM LOCAL '/lessons/dwh/vertica/test_dataset..csv'
DELIMITER '|' ENFORCELENGTH;
```

https://www.vertica.com/docs/11.0.x/HTML/Content/Authoring/SQLReferenceManual/Statements/COPY/COPY.htm?tocpath=SQL%20Reference%20Manual%7CSQL%20Statements%7CCOPY%7C____0

Как писать в Vertica

ИТМО

```
/* для атрибута */
COPY COPY_EXCEL1 (i, v ENFORCELENGTH )
FROM LOCAL 'lessons/dwh/vertica/test_dataset..csv' DELIMITER '|';

/* для всех строковых атрибутов */
COPY COPY_EXCEL1 (i, v)
FROM LOCAL '/lessons/dwh/vertica/test_dataset..csv'
DELIMITER '|' ENFORCELENGTH;


COPY COPY_EXCEL1 (i, v)
FROM LOCAL '/lessons/dwh/vertica/test_dataset..csv'
ABORT ON ERROR
DELIMITER '|'
ENFORCELENGTH;
```

https://www.vertica.com/docs/11.0.x/HTML/Content/Authoring/SQLReferenceManual/Statements/COPY/COPY.htm?tocpath=SQL%20Reference%20Manual%7CSQL%20Statements%7CCOPY%7C____0

Как писать в Vertica

```
COPY COPY_EXCEL1 (i, v ENFORCELENGTH )
FROM LOCAL  'lessons/dwh/vertica/test_dataset..csv''
DELIMITER '|'
REJECTED DATA AS TABLE COPY_EXCEL1_rej ;

# сообщим об ошибке
COPY COPY_EXCEL1 (i, v ENFORCELENGTH )
FROM LOCAL  'lessons/dwh/vertica/test_dataset..csv''
DELIMITER '|'
REJECTED DATA AS TABLE COPY_EXCEL1_rej
REJECTMAX 2 /* 0 = ABORT ON ERROR */ ;
```

https://www.vertica.com/docs/11.0.x/HTML/Content/Authoring/SQLReferenceManual/Statements/COPY/COPY.htm?tocpath=SQL%20Reference%20Manual%7CSQL%20Statements%7CCOPY%7C____0

Как работать с файлами

id, str_characters
1, I like summer, really!
I'm so glad I don't have to go to school!

Другой разделитель

id|str_characters#|I like summer, really! I'm so glad I don't have to go to school!

Экранироание

id, str_characters
1, I like summer\, really!\n I'm so glad I don't have to go to school!

кавычки

id, str_characters
1, “I like summer, really! I'm so glad I don't have to go to school!”

Типы данных

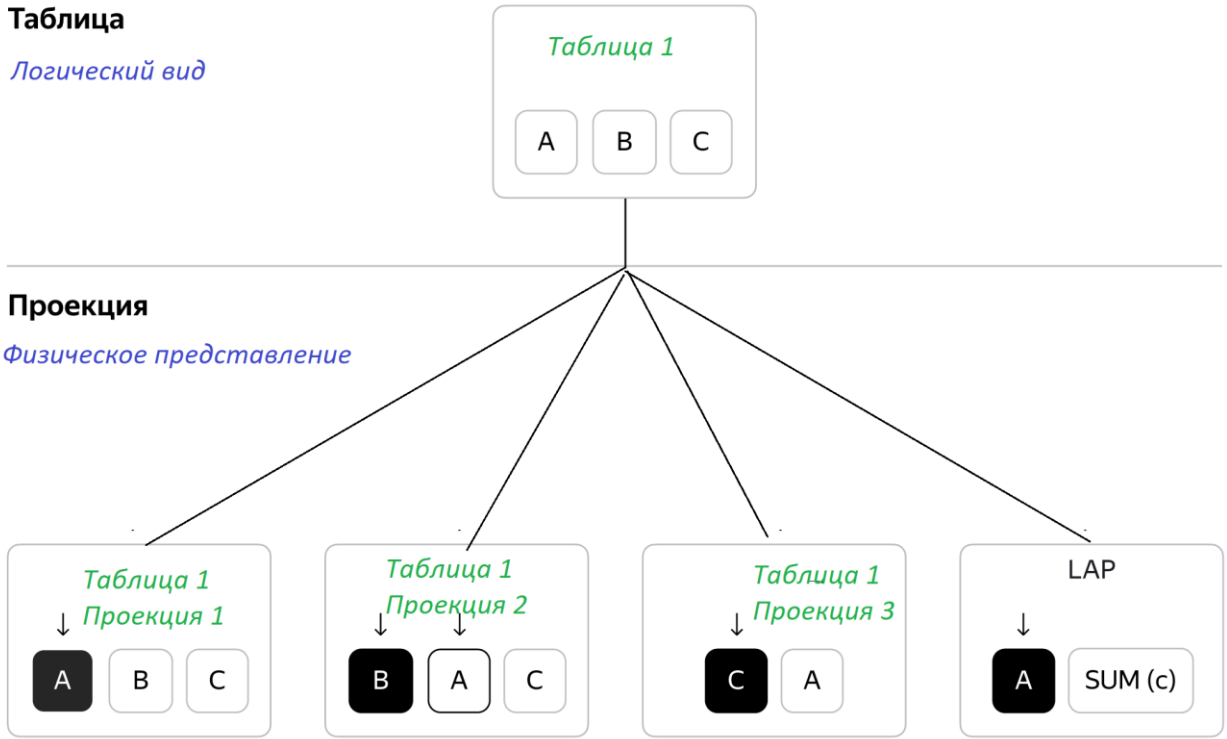
Строковые - VARCHAR(N), LONG VARCHAR и CHAR

Числовые - INTEGER - INT, BIGINT, INT8, SMALLINT и TINYINT

Временные – DATE, TIMESTAMP(N), TIME(N) и TIMETZ(N), INTERVAL

Специальные - LONG VARCHAR,LONG VARBINARY,BOOLEAN,UUID

Проекции, сортировка, сегментация



https://innerlife.io/ru/hp-vertica-k-safety-segmentation-buddy-projection/

Проекции, сортировка, сегментация

```
CREATE TABLE First_table
( i int primary key, v varchar(16) );
INSERT INTO First_table VALUES (1, 'Wonderful day');
select export_objects(' First_table ')

CREATE TABLE {shema_name}. First_table ( i int NOT NULL, v varchar(16), CONSTRAINT
C_PRIMARY PRIMARY KEY (i) DISABLED );

CREATE PROJECTION {shema_name}. First_table /*+createtype(L)*/ ( i, v )
AS
SELECT First_table1.i, First_table.v
FROM {shema_name}. First_table
ORDER BY First_table.i
SEGMENTED BY hash(First_table.i) ALL NODES KSAFE 1;

SELECT MARK_DESIGN_KSAFE(1);
```

Сегментация – несегментируемые проекции

id	data	amount
1	24.05.2024	12
2	24.05.2024	21
3	24.05.2024	34
4	25.05.2024	15
5	25.05.2024	11
6	26.05.2024	10
7	26.05.2024	18
8	26.05.2024	12
9	26.05.2024	9

Нода 1

id	data	amount
1	24.05.2024	12
2	24.05.2024	21
3	24.05.2024	34
4	25.05.2024	15
5	25.05.2024	11
6	26.05.2024	10
7	26.05.2024	18
8	26.05.2024	12
9	26.05.2024	9

Нода 2

id	data	amount
1	24.05.2024	12
2	24.05.2024	21
3	24.05.2024	34
4	25.05.2024	15
5	25.05.2024	11
6	26.05.2024	10
7	26.05.2024	18
8	26.05.2024	12
9	26.05.2024	9

Нода 3

id	data	amount
1	24.05.2024	12
2	24.05.2024	21
3	24.05.2024	34
4	25.05.2024	15
5	25.05.2024	11
6	26.05.2024	10
7	26.05.2024	18
8	26.05.2024	12
9	26.05.2024	9

Сегментация – сегментируемые проекции

	id	data	amount
	1	24.05.2024	12
	2	24.05.2024	21
	3	24.05.2024	34
	4	25.05.2024	15
	5	25.05.2024	11
	6	26.05.2024	10
	7	26.05.2024	18
	8	26.05.2024	12
	9	26.05.2024	9

id	data	amount
3	24.05.2024	34
5	25.05.2024	11
9	26.05.2024	9

id	data	amount
2	24.05.2024	21
4	25.05.2024	15
8	26.05.2024	12

id	data	amount
1	24.05.2024	12
6	26.05.2024	10
7	26.05.2024	18

Сегментация

```
CREATE TABLE Table_1 ( id int PRIMARY KEY, ... name VARCHAR(100) ) ...
SEGMENTED BY HASH(id) ALL NODES;

CREATE TABLE Table_2 ( ...
sender_id int REFERENCES Table_1(id),
receiver_id int REFERENCES Table_1(id), ... ) ...
SEGMENTED BY HASH(sender_id) ALL NODES;
```

Сегментация

```
CREATE TABLE Table_1 ( id int PRIMARY KEY, ... name VARCHAR(100) ) ... SEGMENTED BY HASH(id) ALL NODES;

CREATE TABLE Table_2 ( ...
sender_id int REFERENCES Table_1(id),
receiver_id int REFERENCES Table_1(id), ... ) ...
SEGMENTED BY HASH(sender_id) ALL NODES;


SELECT Table_1.name, count(Table_2.message_id) as sent FROM
Table_2
JOIN Table_1 ON Table_2.sender_id = Table_1.id GROUP BY
Table_1.gender;

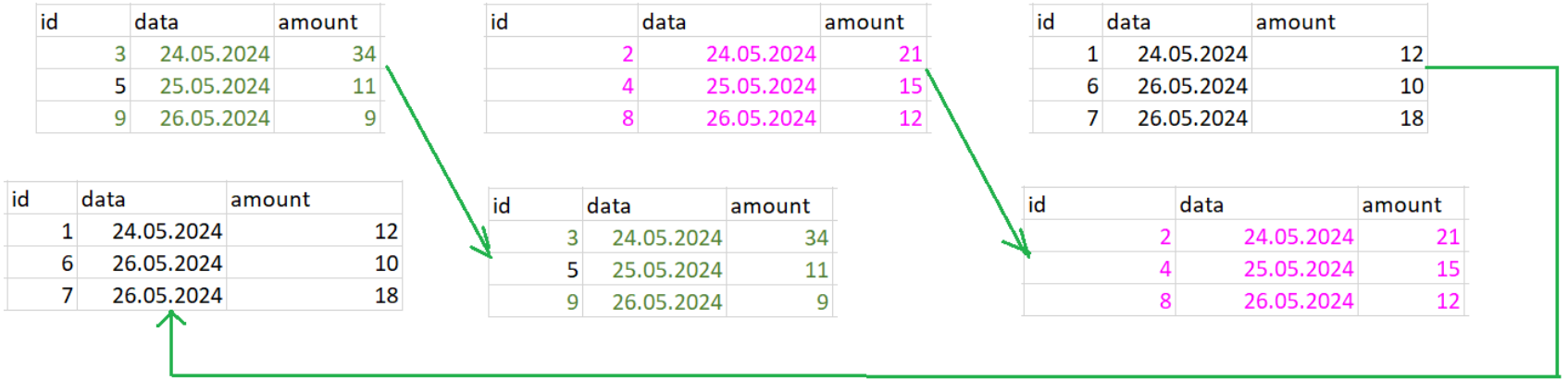
 SELECT Table_1.gender, count(Table_2.message_id) as received
 FROM Table_2
 JOIN Table_1 ON Table_2.receiver_id = Table_1.id
 GROUP BY Table_1.gender;
```

Проекции, сортировка, сегментация

```
CREATE PROJECTION {shema_name}. First_table /*+createtype(L)*/ ( i, v )
AS
SELECT First_table1.i, First_table.v
FROM {shema_name}. First_table
ORDER BY First_table.i
SEGMENTED BY hash(First_table.i) ALL NODES KSAFE 1;

SELECT MARK_DESIGN_KSAFE(1);
```

Проекции, сортировка, сегментация



id	data	amount
3	24.05.2024	34
5	25.05.2024	11
9	26.05.2024	9

id	data	amount
2	24.05.2024	21
4	25.05.2024	15
8	26.05.2024	12

id	data	amount
1	24.05.2024	12
6	26.05.2024	10
7	26.05.2024	18

id	data	amount
1	24.05.2024	12
6	26.05.2024	10
7	26.05.2024	18

id	data	amount
3	24.05.2024	34
5	25.05.2024	11
9	26.05.2024	9

id	data	amount
2	24.05.2024	21
4	25.05.2024	15
8	26.05.2024	12

Удаление и обновление данных

- DELETE
- TRUNCATE
- DROP TABLE

https://bigdataschool.ru/blog/apache-airflow-quick-start.html

Delete

```
CREATE TABLE test ( a int PRIMARY KEY, b int )
ORDER BY a
SEGMENTED BY hash(a) ALL NODES;

CREATE PROJECTION TEST_orderB AS
SELECT a, b
from test
order by b
SEGMENTED BY hash(b) ALL NODES;
```

Tuple Mover

ИТМО

```
WITH ros_per_node as (
select node_name, anchor_table_schema, anchor_table_name,
projection_name, count(*) as ROS_count
from projection_storage
group by 1, 2, 3, 4 )

select anchor_table_schema, anchor_table_name, projection_name,
max(ROS_count) as max_ROS_count
from ros_per_node group by 1, 2, 3
order by max_ROS_count desc, projection_name
```

Эпохи









Ancient History Mark (AHM)

Last Good Epoch (LGE)

Удаление и обновление данных

- DELETE
- TRUNCATE
- UPDATE
- INSERT
- MERGE
  - WHEN MATCHED THEN UPDATE
  - WHEN NOT MATCHED THEN INSERT

Партиционирование

id	data	amount
1	24.05.2015	12
2	24.05.2016	21
3	24.05.2016	34
4	30.05.2016	15
5	30.05.2016	11
6	30.05.2014	10
7	31.05.2016	18
8	31.05.2015	12
9	31.05.2015	9

Сегментация

id	data	amount
1	24.05.2015	12
2	24.05.2016	21
7	31.05.2016	18

id	data	amount
3	24.05.2016	34
6	30.05.2014	10
9	31.05.2015	9

id	data	amount
4	30.05.2016	15
5	30.05.2016	11
8	31.05.2015	12

Партиционирование

2015	id	data	amount
	1	24.05.2015	12

2016	id	data	amount
	2	24.05.2016	21
	7	31.05.2016	18

2014	id	data	amount
	6	30.05.2014	10

2015	id	data	amount
	9	31.05.2015	9

2016	id	data	amount
	3	24.05.2016	34

2015	id	data	amount
	8	31.05.2015	12

2016	id	data	amount
	4	30.05.2016	15
	5	30.05.2016	11

Что можно делать с партициями

Удаление
Разрезание

```
SELECT DROP_PARTITIONS( ‘Schema.Table name',
start partition key,
end partition key
false);
```

Обновление данных
- copy_partitions_to_table(...)
- move_partitions_to_table(...)
- *swap_partitions_between_tables*(...)

Копирование данных
- copy_partitions_to_table()